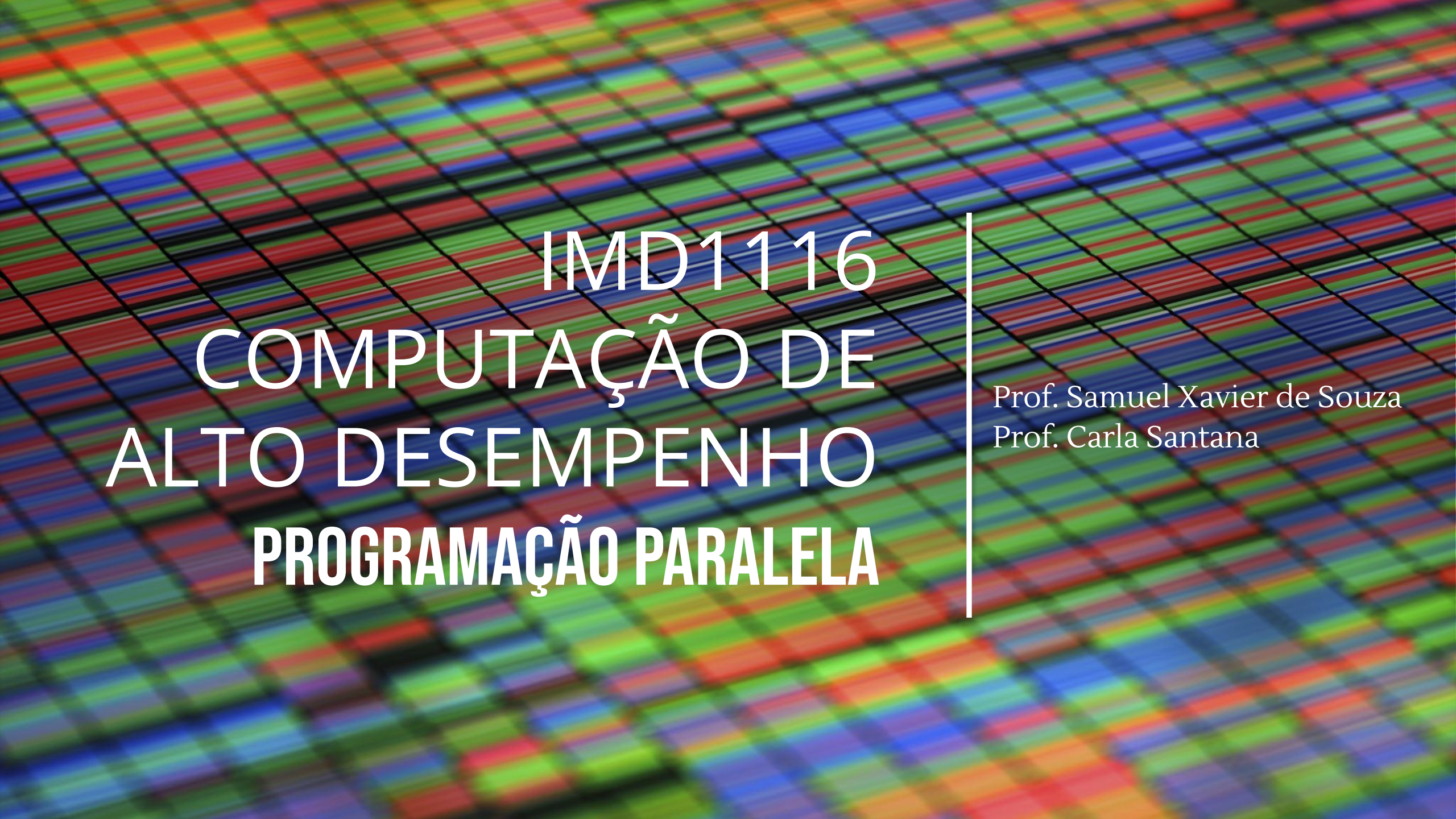




IMD1116

COMPUTAÇÃO DE

ALTO DESEMPENHO

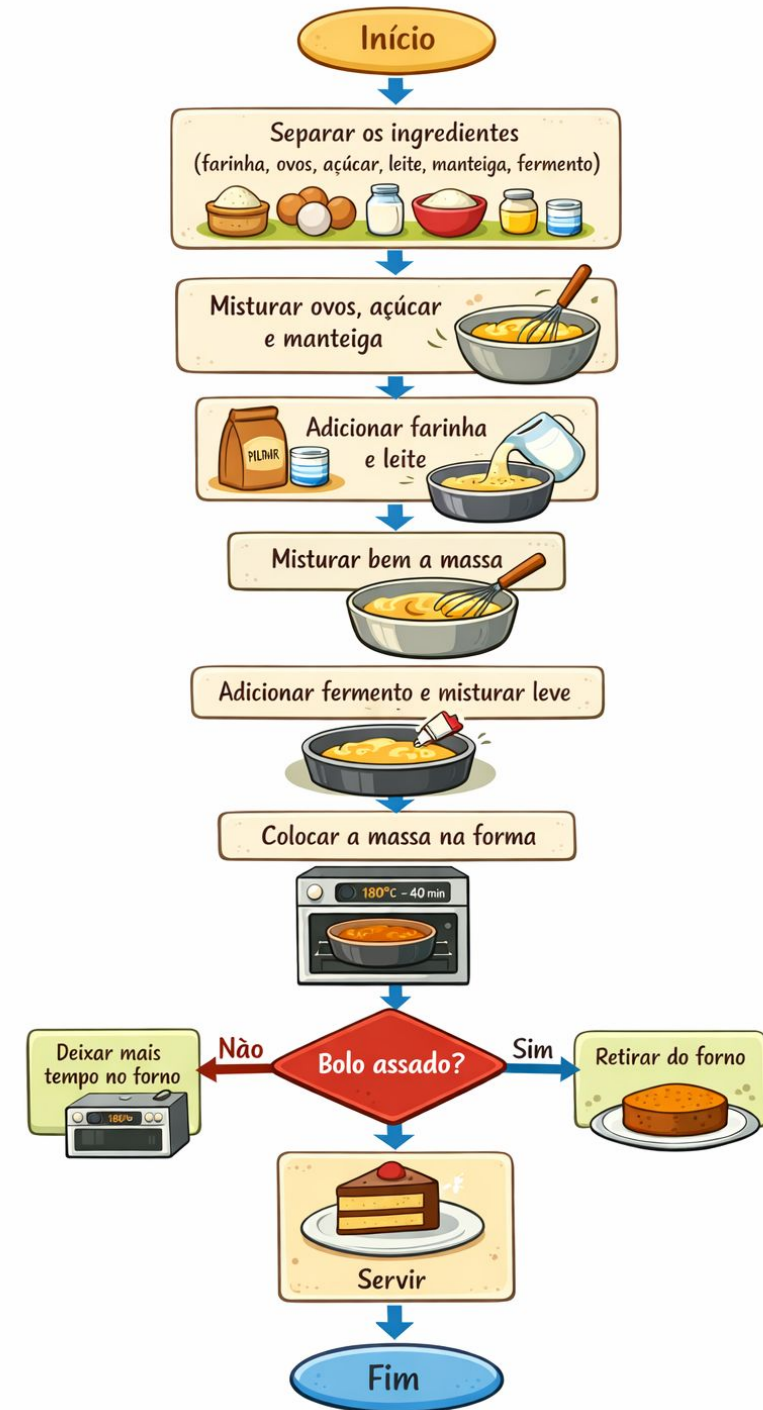
PROGRAMAÇÃO PARALELA

Prof. Samuel Xavier de Souza
Prof. Carla Santana

PROGRAMAÇÃO PARALELA

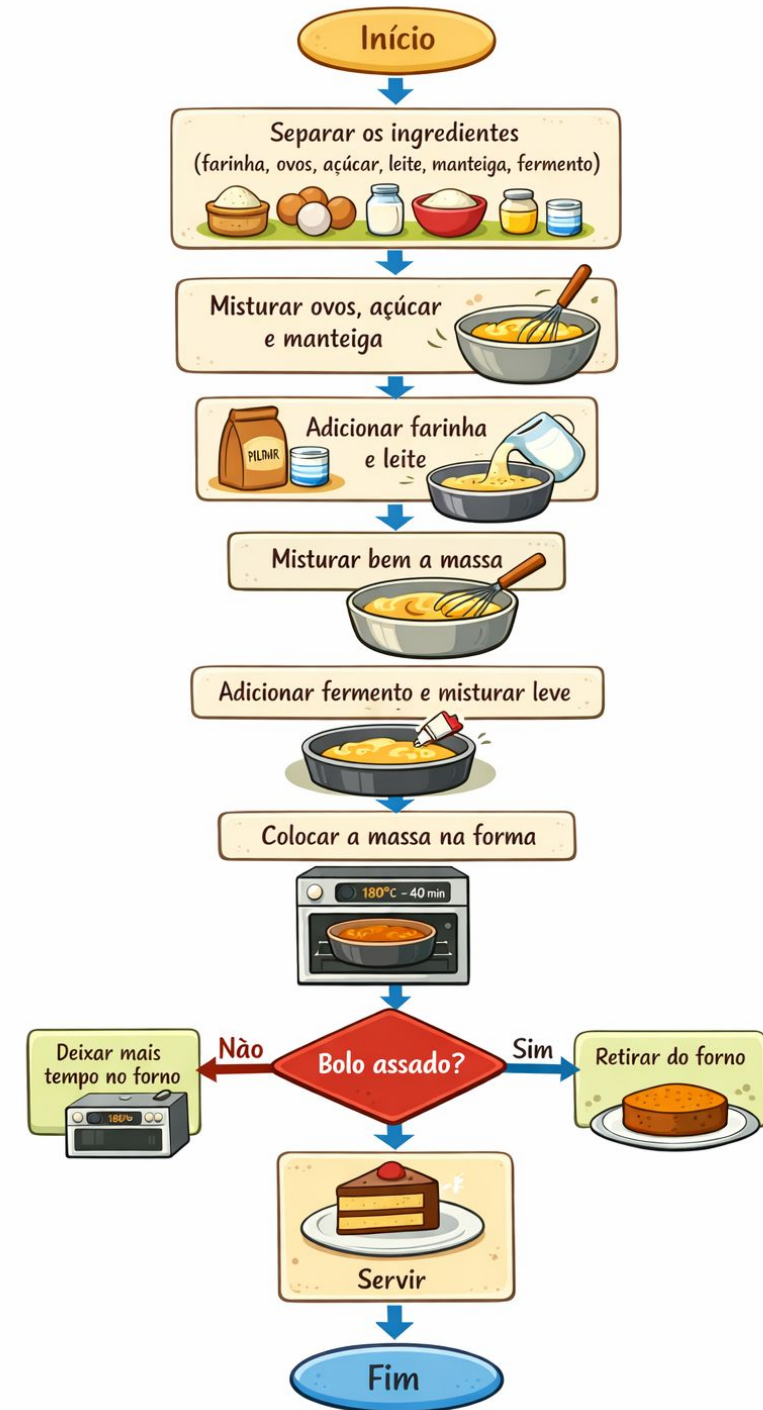
- Objetivos deste tópico
 - Reconhecer a necessidade de maior conhecimento do software para alcançar alto desempenho
 - Reconhecer a maior dificuldade de se programar em paralelo
 - Entender a diferença entre concorrência e paralelismo
 - Identificar a diferença entre memória compartilhada e memória distribuída
 - Introdução a métricas de desempenho
 - Compreensão da limitação por computação e por memória através de uma atividade prática

PROGRAMAÇÃO



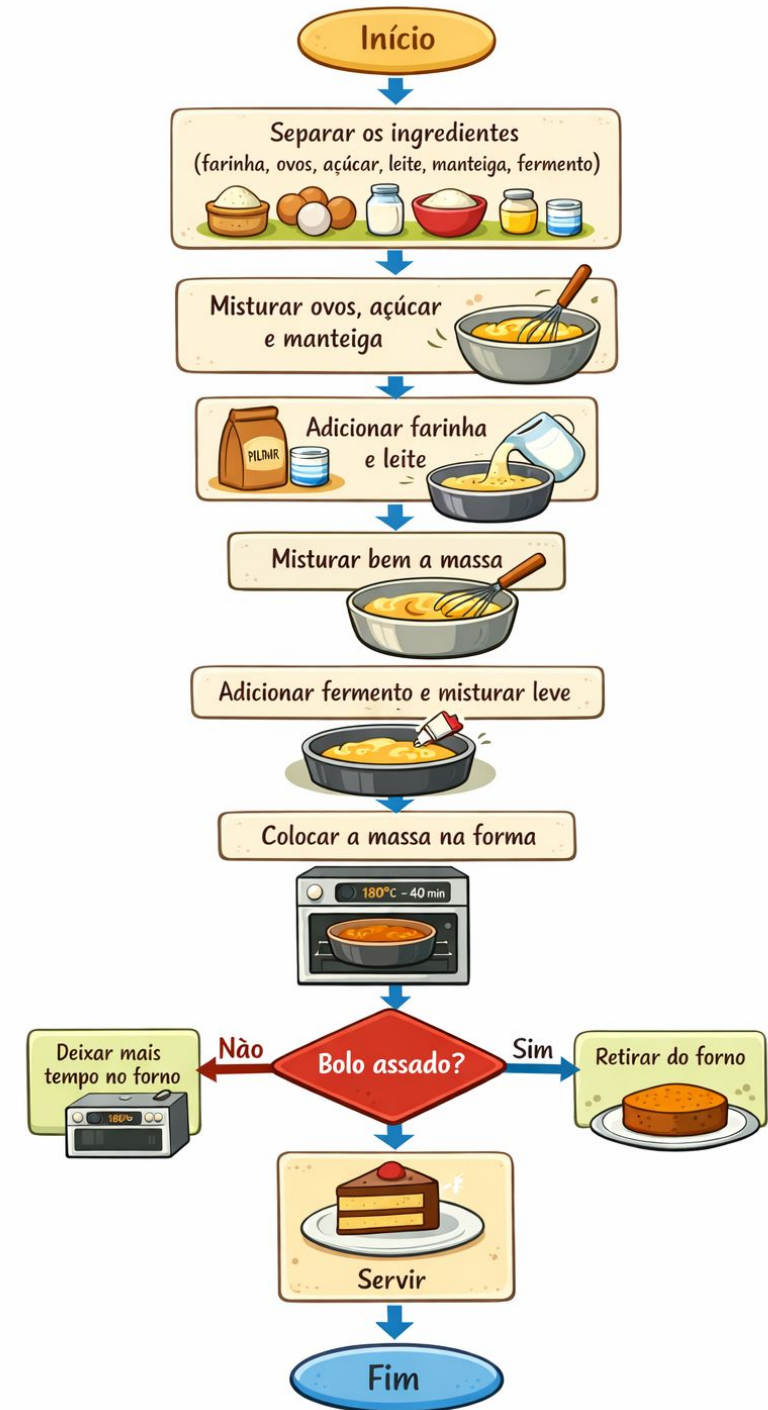
PROGRAMAÇÃO

- Sua receita de bolo para um fluxo só
 - Se você passa a ter mais ajudantes?
 - Seus equipamentos da cozinha melhoram?
- Aplicações são desenvolvidas com um único fluxo de execução (Programação Sequencial)
- Para que a aplicação faça um melhor uso dos recursos de hardware existentes em uma máquina é importante conhecer **o funcionamento das arquiteturas paralelas e consequentemente fazer uso da programação paralela de uma forma mais eficiente** (Rauber and Rünger).



PROGRAMAÇÃO

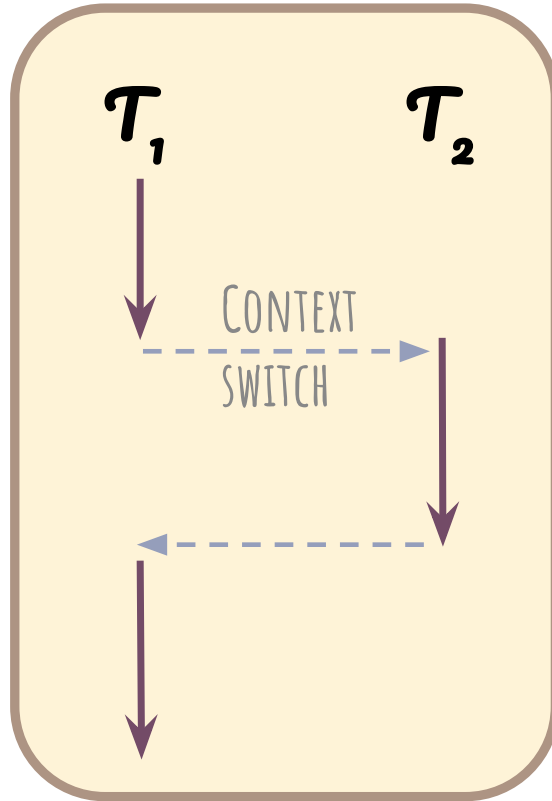
- Qual parte será paralelizada?
 - O que será particionado?
 - Qual paradigma será usado?
 - A versão paralelizada vale a pena?
-



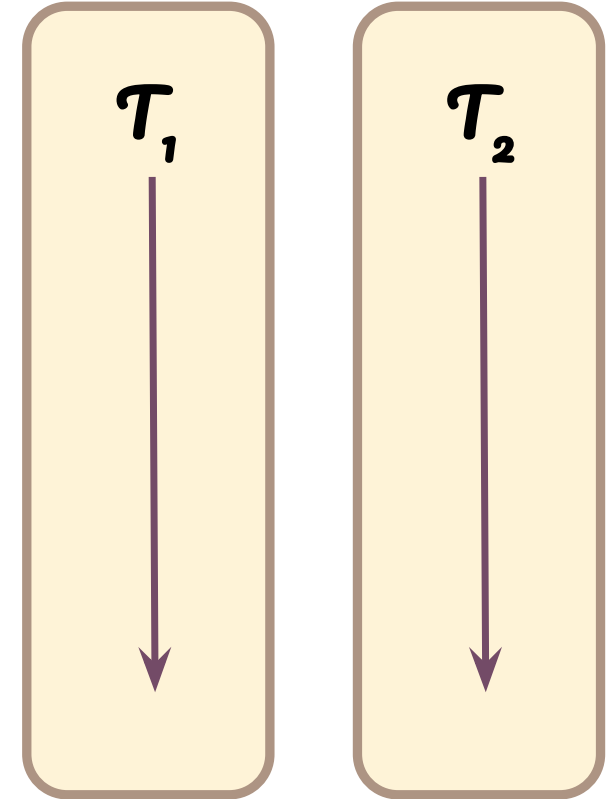
CONCORRÊNCIA X PARALELISMO

CONCORRÊNCIA X PARALELISMO

CONCORRÊNCIA
SINGLE CORE

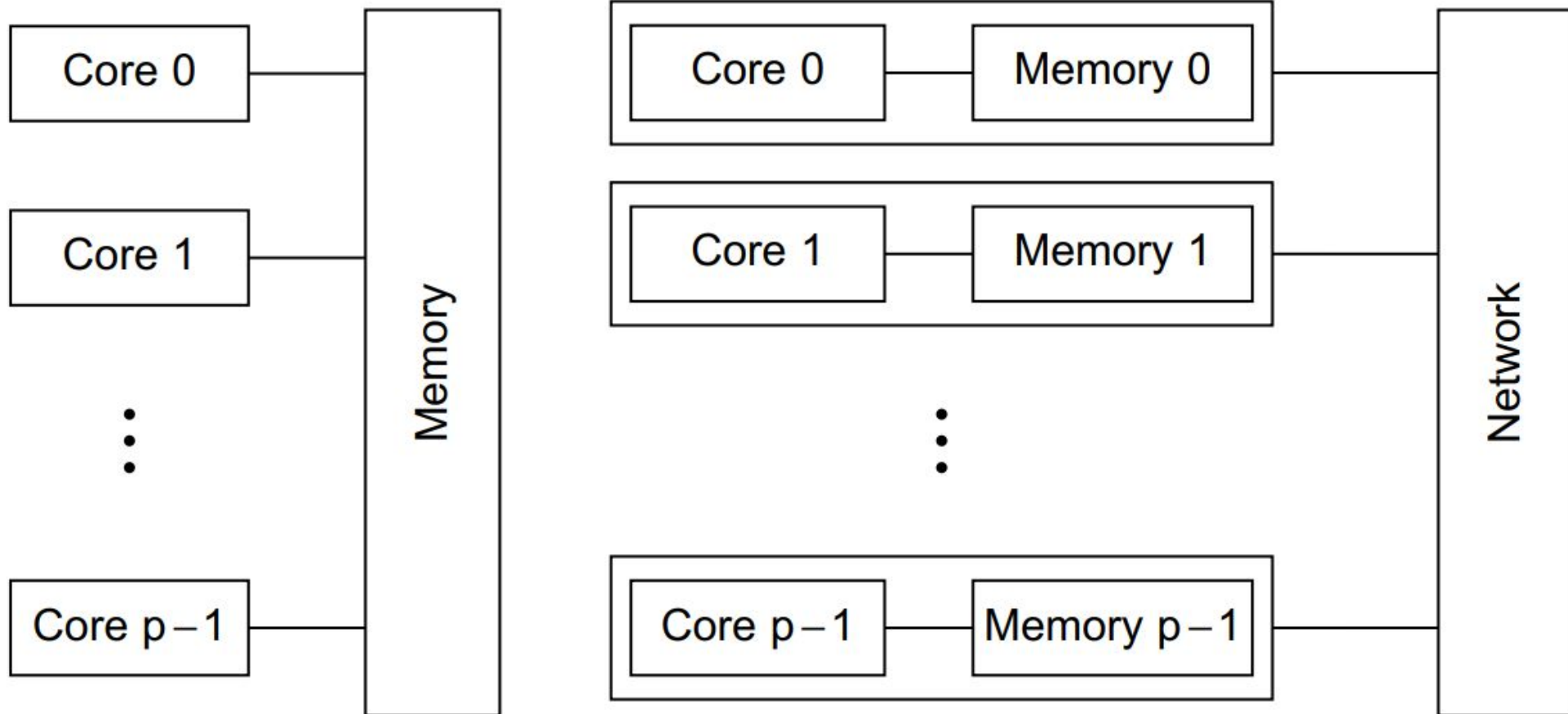


CONCORRÊNCIA
MULTIPLE CORE

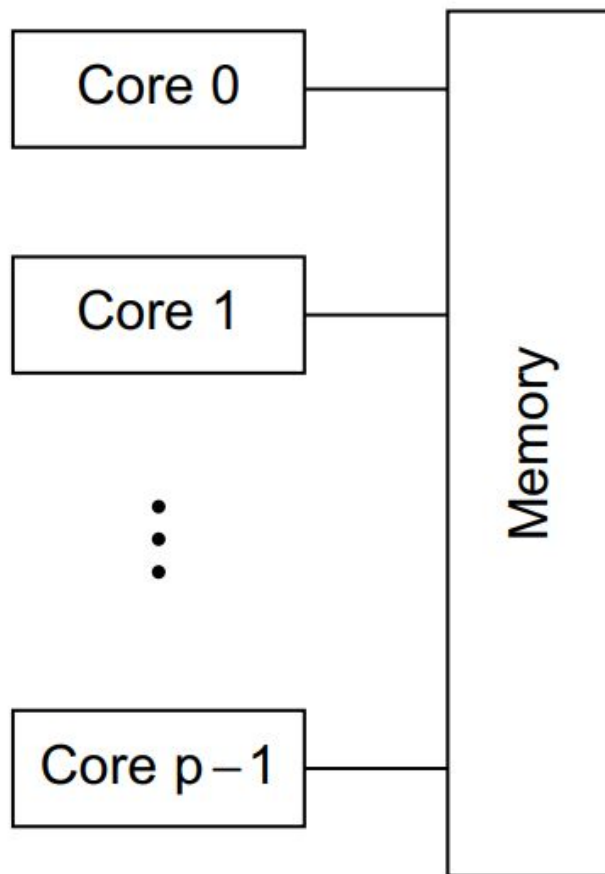


"Duas operações são concorrentes se ambas foram iniciadas e nenhuma foi ainda completada; duas operações são paralelas se realmente podem ser executadas ao mesmo tempo. O paralelismo é, portanto, uma forma de implementação da concorrência."

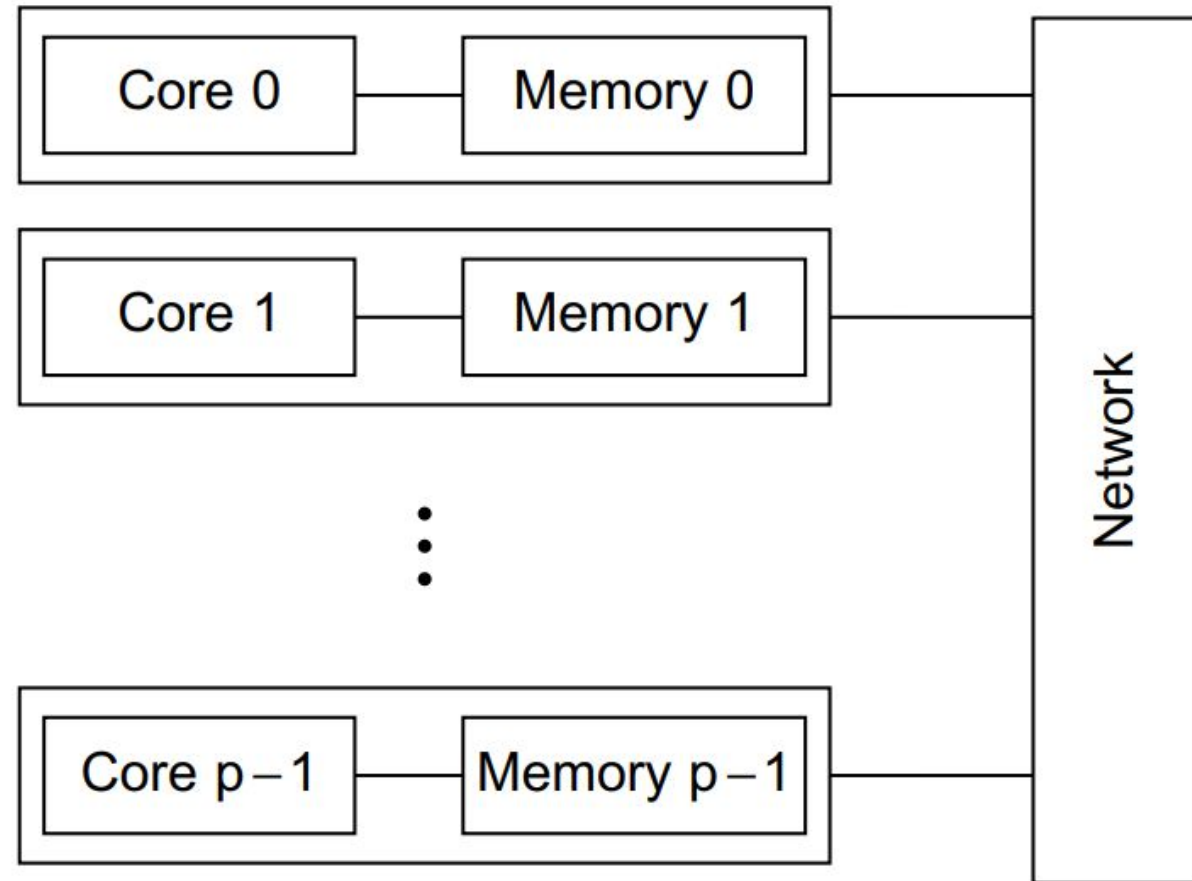
MEMÓRIA COMPARTILHADA X MEMÓRIA DISTRIBUÍDA



MEMÓRIA COMPARTILHADA X MEMÓRIA DISTRIBUÍDA



OPENMP



MPI

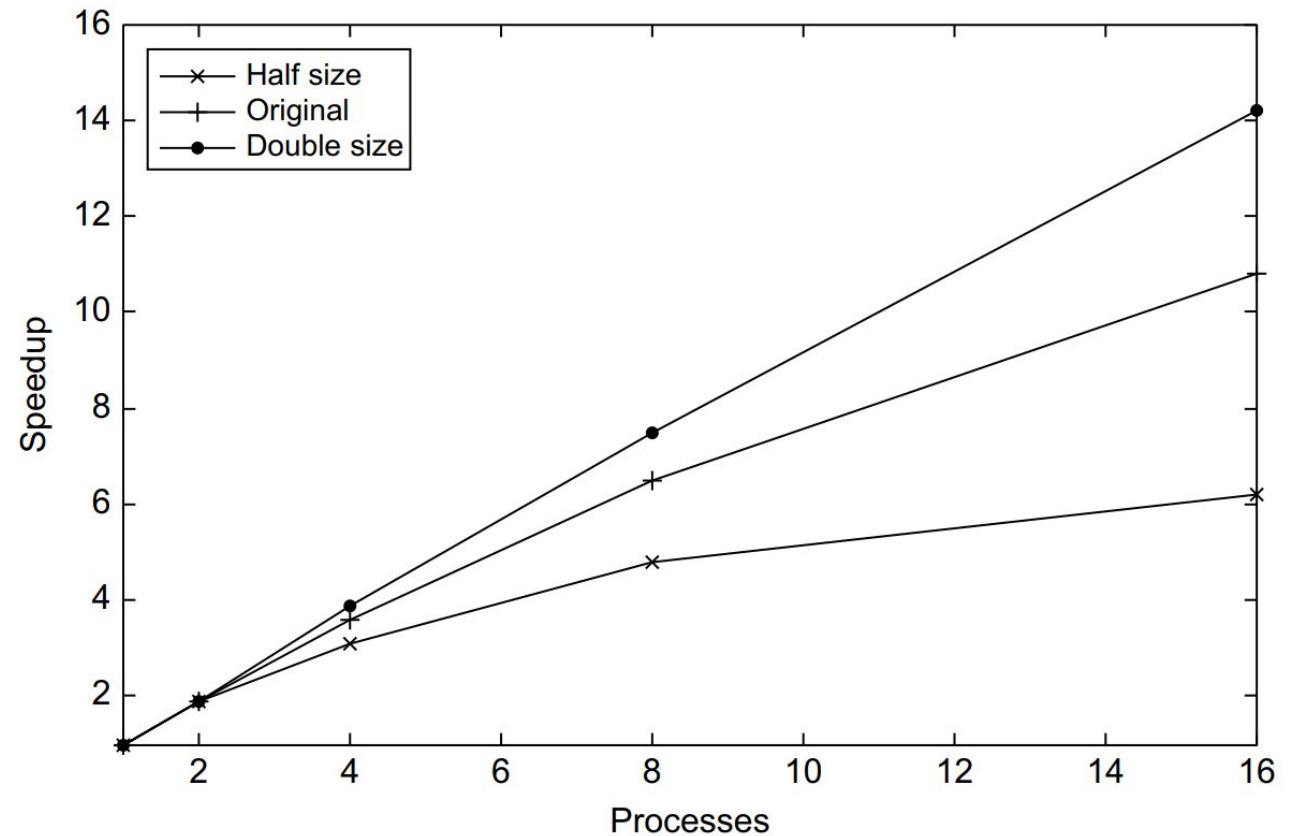
AVALIAÇÃO DE DESEMPENHO

- Fatores que afetam
 - Granularidade da tarefa
 - Mensagens trocadas
 - Dependência entre as tarefas/dados paralelizados
 - Particionamento de Dados X Particionamento de Tarefas
 - Rede de interconexão
 - Trazer os dados para memória
 - Computação nesses dados
- Métricas de Desempenho

Speedup (Aceleração)

$$S(P) = \frac{T(1)}{T(P)}$$

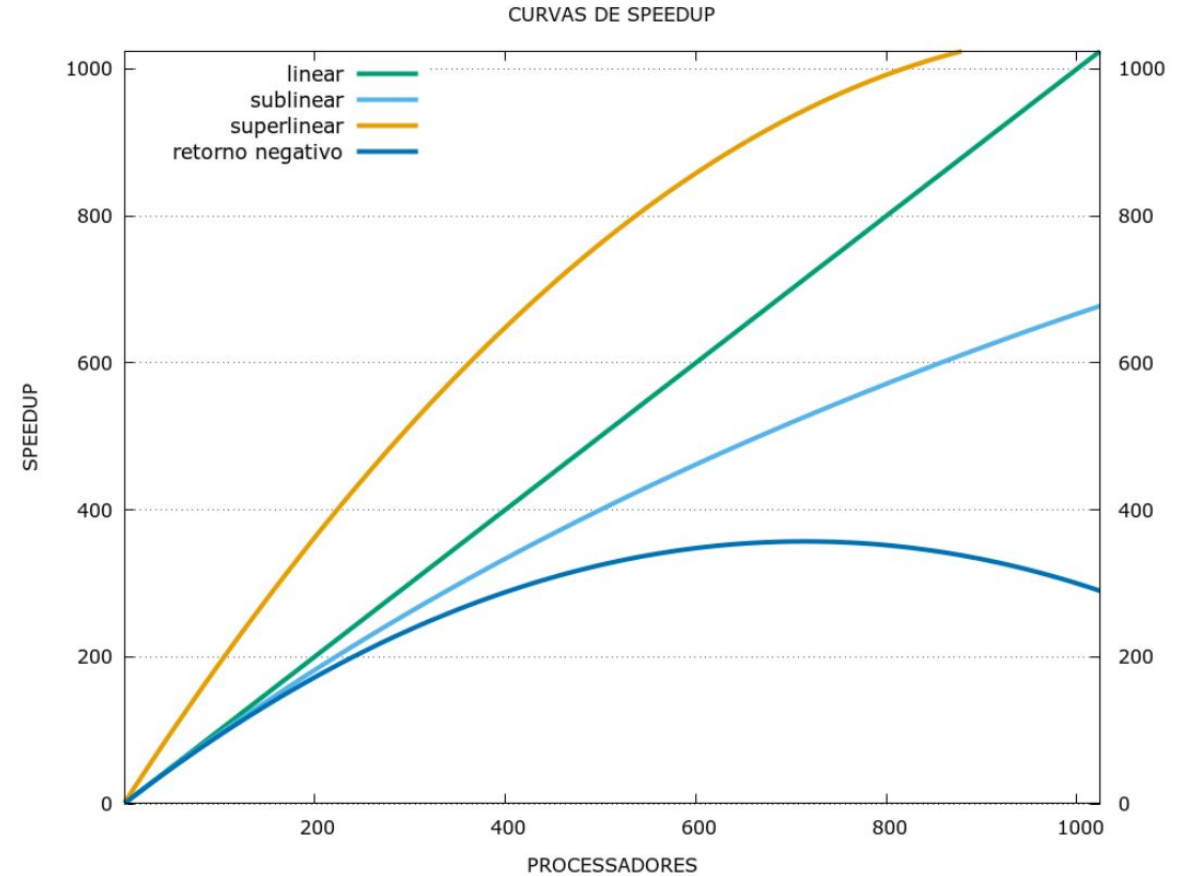
**QUÃO MAIS RÁPIDO FICOU QUANDO SE
COLOCA MAIS PROCESSOS?**



Speedup (Aceleração)

$$S(P) = \frac{T(1)}{T(P)}$$

**QUÃO MAIS RÁPIDO FICOU QUANDO SE
COLOCA MAIS PROCESSOS?**

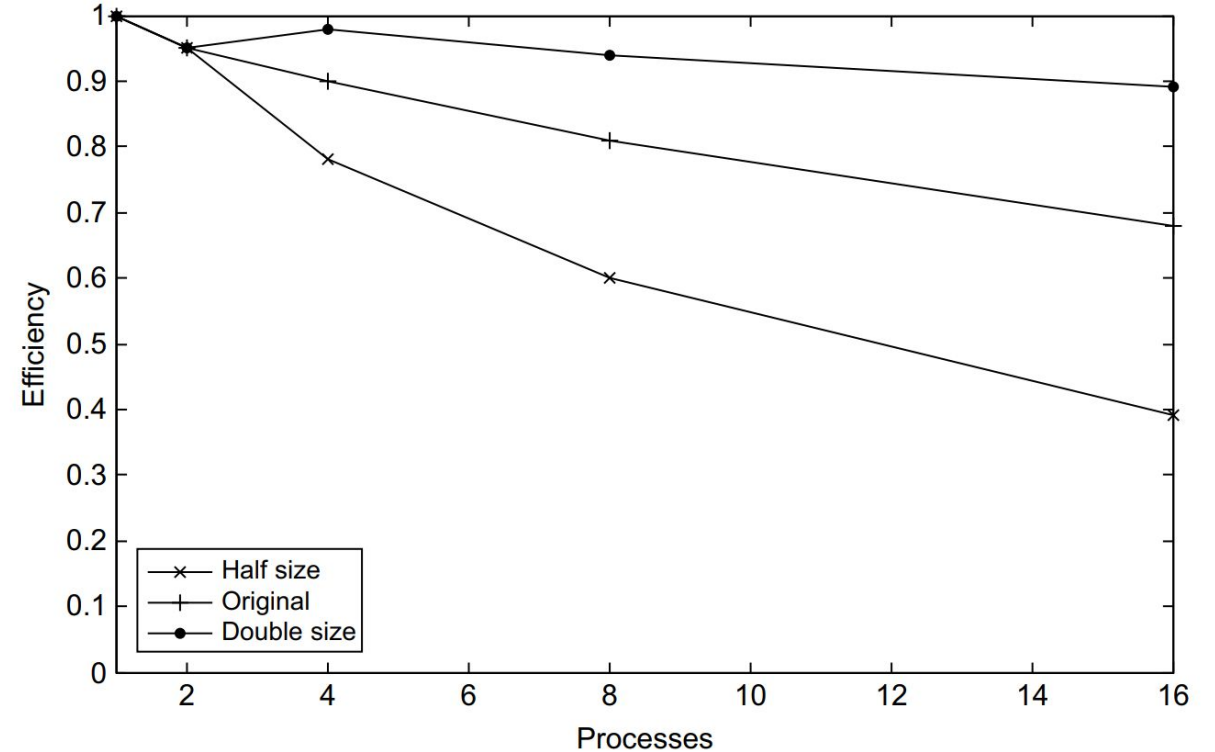


Eficiência

$$E(P) = \frac{S(P)}{P}$$

QUÃO EFETIVA É A ADIÇÃO DE NOVOS PROCESSADORES PARA AJUDAR NA RESOLUÇÃO DE UM PROBLEMA?

SE EU AUMENTAR MEU PROBLEMA A EFICIÊNCIA SE MANTÉM?



MEMORY BOUND X COMPUTE BOUND

	COMPUTAÇÃO ↓	COMPUTAÇÃO ↑
MEMÓRIA ↓		
MEMÓRIA ↑		

MEMORY BOUND X COMPUTE BOUND

	COMPUTAÇÃO ↓	COMPUTAÇÃO ↑
MEMÓRIA ↓	• Subutilização dos recursos • Programa não é pesado o suficiente para estressar o hardware	• Hardware suficiente para lidar com os dados mas não para os cálculos complexos apresentados pelo programa
MEMÓRIA ↑	• Hardware suficiente para lidar com os cálculos mas não com os dados • Data Starvation (fome de dados)	• Hardware insuficiente para lidar com os dados e cálculos • Bottlenecks

MEMORY-BOUND X COMPUTE-BOUND

Tarefa 4:

Implemente **dois programas paralelos em C com OpenMP: um limitado por memória, com somas simples em vetores, e outro limitado por CPU, com cálculos matemáticos intensivos.**

Paralelize com

#pragma omp parallel for
e avalie o desempenho.

Análise quando o desempenho melhora, estabiliza ou piora, e reflita sobre como o multithreading de hardware pode ajudar em programas memory-bound, mas atrapalhar em programas compute-bound pela competição por recursos.

Explique quais são as melhores métricas para avaliar cada caso e como cada uma pode representar os diferentes aspectos do problema.

- Quais programas são limitados pela memória (*gargalo de von Neuman*) e quais não são?
 - Como o multi-thread de hardware pode ajudar (ou atrapalhar)

```
$ gcc -fopenmp meuprog.c -o meuprog
```

```
$ export OMP_NUM_THREADS=2; time meuprog
```

Referências

